

Backend Schema

ApplyWizz Workday Dry-Run Automation Engine V1

1. File Name

backend

2. Product Name

ApplyWizz Workday Dry-Run Automation Engine

3. Purpose of This Document

This document defines the backend data model, authentication architecture, security rules, storage structure, and API expectations for the ApplyWizz Workday Dry-Run Automation Engine V1.

This document should be used before writing:

- Supabase migrations
- database tables
- Row Level Security policies
- storage buckets
- API routes
- worker logic
- authentication logic
- manual review logic
- automation run tracking

The goal is to avoid guessing when building the backend.

4. Backend Stack

The backend should use:

Supabase PostgreSQL
Supabase Auth
Supabase Storage

Node.js Playwright Worker
TypeScript

The frontend will be built with:

Expo React Native + React Native Web

The worker will be separate from the frontend.

5. Core Backend Principle

The backend must support this flow:

```
Candidate
↓
One Resume
↓
Zoho Email
↓
Workday Job Links
↓
Application Runs
↓
Question Extraction
↓
Answer Mapping
↓
Manual Review
↓
Dry-Run Summary
↓
Human Approval
↓
Controlled Submit
```

The backend must never allow uncontrolled auto-submit in V1.

6. Database Provider

Use:

Supabase PostgreSQL

Database type:

PostgreSQL

Primary ID type:

uuid

Timestamp fields:

created_at timestamptz
updated_at timestamptz

All important tables should use UUID primary keys.

7. Authentication Model

Use:

Supabase Auth

V1 login method:

Email + Password

Future login options:

Google OAuth
SSO
Role-based team management

8. User Roles

The system should support these roles:

```
admin
operator
viewer
```

9. Role Permissions

9.1 Admin

Admin can:

- manage users
 - manage candidates
 - upload resumes
 - connect Zoho
 - add job links
 - start dry-runs
 - resolve manual review
 - approve final submit
 - change settings
 - view logs
 - view screenshots
 - retry failed runs
-

9.2 Operator

Operator can:

- create candidates
- edit candidates
- upload resumes
- connect Zoho
- add job links
- start dry-runs
- resolve manual review
- view runs
- view screenshots
- retry failed runs

Operator may or may not approve final submit depending on company policy.

For V1, safest rule:

Only admin can approve final submit.

9.3 Viewer

Viewer can:

- view dashboard
- view candidates
- view job links
- view runs
- view manual review items
- view reports

Viewer cannot:

- create candidates
- upload resumes
- start dry-runs
- edit answers
- approve final submit
- submit applications
- change settings

10. Main Tables

The V1 database should include these tables:

```
users
candidates
candidate_resumes
candidate_answers
question_bank
zoho_mailboxes
workday_accounts
job_links
application_runs
run_steps
extracted_questions
mapped_answers
manual_review_items
otp_logs
screenshots
```

```
automation_logs
system_settings
audit_logs
```

11. Table: users

Purpose

Stores internal ApplyWizz users who can access the dashboard.

Supabase Auth stores the actual login account. This table stores app-specific profile and role data.

Columns

```
id uuid primary key references auth.users(id) on delete cascade
full_name text not null
email text not null unique
role text not null check role in ('admin', 'operator', 'viewer')
status text not null default 'active' check status in ('active', 'inactive',
'suspended')
avatar_url text null
last_login_at timestampz null
created_at timestampz not null default now()
updated_at timestampz not null default now()
```

Indexes

```
users_email_idx on users(email)
users_role_idx on users(role)
users_status_idx on users(status)
```

RLS

- Admin can read all users.
- Operator can read their own profile.
- Viewer can read their own profile.
- Only admin can update roles.
- Users can update limited personal fields like full_name and avatar_url.

12. Table: candidates

Purpose

Stores candidate profile information.

Each candidate has one active resume in V1.

Columns

```
id uuid primary key default gen_random_uuid()
created_by uuid references users(id) null
full_name text not null
email text not null
phone text null
location text null
target_role text null
years_experience numeric null
linkedin_url text null
github_url text null
portfolio_url text null
current_company text null
expected_salary text null
availability text null
relocation_preference text null
sponsorship_requirement text null
work_authorization text null
status text not null default 'active' check status in ('active', 'inactive',
'archived')
notes text null
created_at timestamptz not null default now()
updated_at timestamptz not null default now()
```

Indexes

```
candidates_email_idx on candidates(email)
candidates_status_idx on candidates(status)
candidates_created_by_idx on candidates(created_by)
candidates_target_role_idx on candidates(target_role)
```

RLS

- Admin can read/write all candidates.

- Operator can read/write candidates.
- Viewer can read candidates only.
- No public access.

13. Table: candidate_resumes

Purpose

Stores resume metadata. Actual files are stored in Supabase Storage.

V1 rule:

Only one active resume per candidate.

Columns

```
id uuid primary key default gen_random_uuid()
candidate_id uuid not null references candidates(id) on delete cascade
storage_bucket text not null default 'candidate-resumes'
storage_path text not null
file_name text not null
file_type text not null
file_size_bytes bigint null
is_active boolean not null default true
uploaded_by uuid references users(id) null
notes text null
created_at timestamptz not null default now()
updated_at timestamptz not null default now()
```

Indexes

```
candidate_resumes_candidate_id_idx on candidate_resumes(candidate_id)
candidate_resumes_is_active_idx on candidate_resumes(is_active)
candidate_resumes_candidate_active_idx on candidate_resumes(candidate_id,
is_active)
```

Unique Constraint

Only one active resume per candidate:


```
unique partial index on candidate_resumes(candidate_id) where is_active = true
```

RLS

- Admin/operator can upload and manage resumes.
- Viewer can view metadata only.
- Resume files must be accessed using signed URLs.
- No public resume access.

14. Table: zoho_mailboxes

Purpose

Stores Zoho mailbox connection metadata for OTP reading.

Sensitive tokens must be encrypted.

Columns

```
id uuid primary key default gen_random_uuid()
candidate_id uuid not null references candidates(id) on delete cascade
email text not null
zoho_account_id text null
access_token_encrypted text null
refresh_token_encrypted text null
token_expires_at timestampz null
connection_status text not null default 'not_connected'
    check connection_status in ('not_connected', 'connected', 'expired', 'failed',
    'revoked')
last_otp_check_at timestampz null
last_success_at timestampz null
last_error text null
created_at timestampz not null default now()
updated_at timestampz not null default now()
```

Indexes

```
zoho_mailboxes_candidate_id_idx on zoho_mailboxes(candidate_id)
zoho_mailboxes_email_idx on zoho_mailboxes(email)
zoho_mailboxes_status_idx on zoho_mailboxes(connection_status)
```

Sensitive Fields

Encrypt:

```
access_token_encrypted  
refresh_token_encrypted
```

RLS

- Admin/operator can view connection status.
 - Tokens must never be exposed to frontend.
 - Worker can access encrypted tokens through service role only.
 - Viewer cannot see token fields.
-

15. Table: `workday_accounts`

Purpose

Stores Workday tenant login/account metadata per candidate.

One candidate may have many Workday accounts because each company can have a different Workday tenant.

Columns

```
id uuid primary key default gen_random_uuid()  
candidate_id uuid not null references candidates(id) on delete cascade  
tenant_key text not null  
tenant_name text null  
workday_base_url text null  
email text not null  
username text null  
password_encrypted text null  
account_status text not null default 'unknown'  
  check account_status in ('unknown', 'created', 'existing', 'login_success',  
  'login_failed', 'otp_required', 'locked', 'disabled')  
last_login_at timestamptz null  
last_error text null  
created_at timestamptz not null default now()  
updated_at timestamptz not null default now()
```

Indexes

```
workday_accounts_candidate_id_idx on workday_accounts(candidate_id)
workday_accounts_tenant_key_idx on workday_accounts(tenant_key)
workday_accounts_candidate_tenant_idx on workday_accounts(candidate_id,
tenant_key)
workday_accounts_status_idx on workday_accounts(account_status)
```

Unique Constraint

```
unique(candidate_id, tenant_key)
```

Sensitive Fields

Encrypt:

```
password_encrypted
```

RLS

- Admin/operator can view account status.
 - Password must never be exposed to frontend.
 - Worker can decrypt only in secure runtime.
 - Viewer cannot access sensitive fields.
-

16. Table: job_links

Purpose

Stores Workday job links that need to be processed.

Columns

```
id uuid primary key default gen_random_uuid()
candidate_id uuid not null references candidates(id) on delete cascade
created_by uuid references users(id) null
url text not null
normalized_url text not null
company_name text null
```

```

job_title text null
workday_tenant_key text null
source text null
status text not null default 'queued'
    check status in (
        'queued',
        'running',
        'opened',
        'login_required',
        'otp_required',
        'logged_in',
        'resume_uploaded',
        'questionnaire_reached',
        'questions_extracted',
        'answers_mapped',
        'manual_review_required',
        'dry_run_complete',
        'approved_for_submit',
        'submitted',
        'failed',
        'duplicate',
        'skipped'
    )
last_run_id uuid null
last_error text null
priority integer not null default 0
notes text null
created_at timestamptz not null default now()
updated_at timestamptz not null default now()

```

Indexes

```

job_links_candidate_id_idx on job_links(candidate_id)
job_links_status_idx on job_links(status)
job_links_normalized_url_idx on job_links(normalized_url)
job_links_workday_tenant_key_idx on job_links(workday_tenant_key)
job_links_created_at_idx on job_links(created_at)
job_links_priority_idx on job_links(priority)

```

Unique Constraint

Avoid duplicate job links per candidate:

```
unique(candidate_id, normalized_url)
```

RLS

- Admin/operator can create and update.
- Viewer can read.
- No public access.

17. Table: application_runs

Purpose

Stores each automation attempt.

One job link can have multiple runs if retried.

Columns

```
id uuid primary key default gen_random_uuid()
job_link_id uuid not null references job_links(id) on delete cascade
candidate_id uuid not null references candidates(id) on delete cascade
started_by uuid references users(id) null
status text not null default 'queued'
  check status in (
    'queued',
    'starting',
    'opening_job_link',
    'detecting_page_state',
    'login_or_create_account',
    'waiting_for_otp',
    'otp_verified',
    'uploading_resume',
    'resume_uploaded',
    'reaching_questionnaire',
    'capturing_html',
    'extracting_questions',
    'mapping_answers',
    'filling_safe_answers',
    'checking_conditional_questions',
    'manual_review_required',
    'dry_run_complete',
    'human_approval_required',
```

```

        'approved_for_submit',
        'submitting',
        'submitted',
        'stopped',
        'failed'
    )
mode text not null default 'dry_run' check mode in ('dry_run',
'controlled_submit')
current_step text null
readiness_score text null check readiness_score in ('ready', 'needs_review',
'blocked', 'failed')
total_questions_found integer not null default 0
total_answers_mapped integer not null default 0
total_answers_filled integer not null default 0
total_manual_review_items integer not null default 0
total_high_risk_items integer not null default 0
error_code text null
error_message text null
started_at timestamptz null
completed_at timestamptz null
approved_by uuid references users(id) null
approved_at timestamptz null
submitted_at timestamptz null
created_at timestamptz not null default now()
updated_at timestamptz not null default now()

```

Indexes

```

application_runs_job_link_id_idx on application_runs(job_link_id)
application_runs_candidate_id_idx on application_runs(candidate_id)
application_runs_status_idx on application_runs(status)
application_runs_started_by_idx on application_runs(started_by)
application_runs_created_at_idx on application_runs(created_at)
application_runs_readiness_score_idx on application_runs(readiness_score)

```

RLS

- Admin/operator can create runs.
- Admin/operator can read and update run status through approved actions.
- Worker updates run status using service role.
- Viewer can read only.

18. Table: run_steps

Purpose

Stores step-by-step timeline for each automation run.

Columns

```
id uuid primary key default gen_random_uuid()
application_run_id uuid not null references application_runs(id) on delete
cascade
step_name text not null
step_status text not null default 'started'
    check step_status in ('started', 'success', 'failed', 'skipped', 'waiting',
'retrying')
step_order integer not null
message text null
current_url text null
duration_ms integer null
error_code text null
error_message text null
metadata jsonb null
started_at timestamptz null
completed_at timestamptz null
created_at timestamptz not null default now()
```

Indexes

```
run_steps_application_run_id_idx on run_steps(application_run_id)
run_steps_step_name_idx on run_steps(step_name)
run_steps_step_status_idx on run_steps(step_status)
run_steps_order_idx on run_steps(application_run_id, step_order)
```

RLS

- Admin/operator/viewer can read.
- Worker can write using service role.
- Frontend should not directly create technical run steps.

19. Table: extracted_questions

Purpose

Stores questions extracted from Workday forms.

Columns

```
id uuid primary key default gen_random_uuid()
application_run_id uuid not null references application_runs(id) on delete
cascade
job_link_id uuid not null references job_links(id) on delete cascade
candidate_id uuid not null references candidates(id) on delete cascade
question_text text not null
normalized_question text not null
section_name text null
field_type text not null
    check field_type in (
        'text',
        'number',
        'email',
        'phone',
        'date',
        'dropdown',
        'radio',
        'checkbox',
        'multi_select',
        'yes_no',
        'file_upload',
        'address',
        'legal_acknowledgment',
        'unknown'
    )
field_name text null
field_id text null
field_selector text null
options jsonb null
is_required boolean not null default false
is_visible boolean not null default true
page_step text null
risk_level text not null default 'unknown'
    check risk_level in ('low', 'medium', 'high', 'unknown')
extraction_confidence numeric null
source_html_storage_path text null
```



```
screenshot_id uuid null
created_at timestamptz not null default now()
```

Indexes

```
extracted_questions_application_run_id_idx on
extracted_questions(application_run_id)
extracted_questions_job_link_id_idx on extracted_questions(job_link_id)
extracted_questions_candidate_id_idx on extracted_questions(candidate_id)
extracted_questions_normalized_question_idx on
extracted_questions(normalized_question)
extracted_questions_risk_level_idx on extracted_questions(risk_level)
extracted_questions_field_type_idx on extracted_questions(field_type)
```

RLS

- Admin/operator/viewer can read.
- Worker writes using service role.
- Operators can update risk classification only through controlled review actions.

20. Table: candidate_answers

Purpose

Stores candidate-specific approved answers.

These are used before global answers because candidate-specific answers are more accurate.

Columns

```
id uuid primary key default gen_random_uuid()
candidate_id uuid not null references candidates(id) on delete cascade
question_key text not null
question_text text null
normalized_question text null
answer_value text not null
answer_json jsonb null
answer_type text not null default 'text'
    check answer_type in ('text', 'number', 'boolean', 'single_select',
    'multi_select', 'date', 'file', 'json')
risk_level text not null default 'unknown'
```

```
    check risk_level in ('low', 'medium', 'high', 'unknown')
category text null
is_approved boolean not null default false
approved_by uuid references users(id) null
approved_at timestampz null
source text null
notes text null
created_at timestampz not null default now()
updated_at timestampz not null default now()
```

Indexes

```
candidate_answers_candidate_id_idx on candidate_answers(candidate_id)
candidate_answers_question_key_idx on candidate_answers(question_key)
candidate_answers_normalized_question_idx on
candidate_answers(normalized_question)
candidate_answers_risk_level_idx on candidate_answers(risk_level)
candidate_answers_is_approved_idx on candidate_answers(is_approved)
```

Unique Constraint

```
unique(candidate_id, question_key)
```

RLS

- Admin/operator can read and write.
- Viewer can read.
- High-risk answers require approval.
- AI must not create approved high-risk answers automatically.

21. Table: question_bank

Purpose

Stores reusable global Workday question patterns and approved mapping rules.

Columns

```
id uuid primary key default gen_random_uuid()
question_key text not null unique
```

```

canonical_question text not null
normalized_question text not null
question_variants jsonb null
answer_type text not null default 'text'
    check answer_type in ('text', 'number', 'boolean', 'single_select',
'multi_select', 'date', 'file', 'json')
default_answer_value text null
default_answer_json jsonb null
risk_level text not null default 'unknown'
    check risk_level in ('low', 'medium', 'high', 'unknown')
category text null
is_global boolean not null default true
is_active boolean not null default true
requires_manual_review boolean not null default true
usage_count integer not null default 0
success_count integer not null default 0
last_used_at timestamptz null
created_by uuid references users(id) null
approved_by uuid references users(id) null
approved_at timestamptz null
created_at timestamptz not null default now()
updated_at timestamptz not null default now()

```

Indexes

```

question_bank_question_key_idx on question_bank(question_key)
question_bank_normalized_question_idx on question_bank(normalized_question)
question_bank_risk_level_idx on question_bank(risk_level)
question_bank_category_idx on question_bank(category)
question_bank_is_active_idx on question_bank(is_active)

```

RLS

- Admin can create and approve global rules.
- Operator can suggest or create candidate-specific answers.
- Viewer can read.
- High-risk global answers should not be auto-approved.

22. Table: mapped_answers

Purpose

Stores the mapping result between extracted questions and selected answers.

Columns

```
id uuid primary key default gen_random_uuid()
application_run_id uuid not null references application_runs(id) on delete
cascade
extracted_question_id uuid not null references extracted_questions(id) on delete
cascade
candidate_id uuid not null references candidates(id) on delete cascade
source_type text not null
    check source_type in ('candidate_answer', 'question_bank', 'profile_field',
'manual_review', 'ai_suggested', 'none')
source_id uuid null
suggested_answer text null
suggested_answer_json jsonb null
final_answer text null
final_answer_json jsonb null
confidence_score numeric null
risk_level text not null default 'unknown'
    check risk_level in ('low', 'medium', 'high', 'unknown')
mapping_status text not null default 'pending'
    check mapping_status in ('pending', 'mapped', 'needs_review', 'approved',
'rejected', 'filled', 'skipped')
requires_manual_review boolean not null default true
ai_reasoning_summary text null
approved_by uuid references users(id) null
approved_at timestamptz null
filled_at timestamptz null
created_at timestamptz not null default now()
updated_at timestamptz not null default now()
```

Indexes

```
mapped_answers_application_run_id_idx on mapped_answers(application_run_id)
mapped_answers_extracted_question_id_idx on
mapped_answers(extracted_question_id)
mapped_answers_candidate_id_idx on mapped_answers(candidate_id)
mapped_answers_mapping_status_idx on mapped_answers(mapping_status)
mapped_answers_risk_level_idx on mapped_answers(risk_level)
mapped_answers_requires_manual_review_idx on
mapped_answers(requires_manual_review)
```

RLS

- Admin/operator can review and approve.

- Viewer can read.
- Worker can write mapping results using service role.
- AI-suggested answers are not final until approved or low-risk confidence gate passes.

23. Table: manual_review_items

Purpose

Stores questions and answers requiring human review.

Columns

```
id uuid primary key default gen_random_uuid()
application_run_id uuid not null references application_runs(id) on delete
cascade
candidate_id uuid not null references candidates(id) on delete cascade
job_link_id uuid not null references job_links(id) on delete cascade
extracted_question_id uuid not null references extracted_questions(id) on delete
cascade
mapped_answer_id uuid null references mapped_answers(id) on delete set null
review_reason text not null
risk_level text not null default 'unknown'
    check risk_level in ('low', 'medium', 'high', 'unknown')
question_text text not null
field_type text null
options jsonb null
suggested_answer text null
final_answer text null
status text not null default 'open'
    check status in ('open', 'approved', 'edited', 'rejected', 'marked_unsafe',
'skipped', 'resolved')
assigned_to uuid references users(id) null
resolved_by uuid references users(id) null
resolved_at timestampz null
save_to_answer_bank boolean not null default false
created_at timestampz not null default now()
updated_at timestampz not null default now()
```

Indexes

```
manual_review_items_application_run_id_idx on
manual_review_items(application_run_id)
```

```
manual_review_items_candidate_id_idx on manual_review_items(candidate_id)
manual_review_items_job_link_id_idx on manual_review_items(job_link_id)
manual_review_items_status_idx on manual_review_items(status)
manual_review_items_risk_level_idx on manual_review_items(risk_level)
manual_review_items_assigned_to_idx on manual_review_items(assigned_to)
```

RLS

- Admin/operator can resolve review items.
- Viewer can read only.
- High-risk review items require explicit human action.

24. Table: otp_logs

Purpose

Stores OTP detection and usage events.

Do not store full email content.

Columns

```
id uuid primary key default gen_random_uuid()
candidate_id uuid not null references candidates(id) on delete cascade
application_run_id uuid null references application_runs(id) on delete cascade
zoho_mailbox_id uuid null references zoho_mailboxes(id) on delete set null
provider text not null default 'zoho'
email_subject text null
sender_email text null
otp_code_masked text null
otp_found boolean not null default false
used_successfully boolean not null default false
error_message text null
checked_at timestampz not null default now()
created_at timestampz not null default now()
```

Indexes

```
otp_logs_candidate_id_idx on otp_logs(candidate_id)
otp_logs_application_run_id_idx on otp_logs(application_run_id)
otp_logs_zoho_mailbox_id_idx on otp_logs(zoho_mailbox_id)
```

```
otp_logs_checked_at_idx on otp_logs(checked_at)
otp_logs_otp_found_idx on otp_logs(otp_found)
```

Privacy Rule

Store masked OTP only:

```
***** or last 2 digits only
```

Do not store full OTP unless absolutely required. Prefer not to store it.

25. Table: screenshots

Purpose

Stores screenshot metadata. Actual image files are stored in Supabase Storage.

Columns

```
id uuid primary key default gen_random_uuid()
application_run_id uuid not null references application_runs(id) on delete
cascade
run_step_id uuid null references run_steps(id) on delete set null
candidate_id uuid not null references candidates(id) on delete cascade
job_link_id uuid not null references job_links(id) on delete cascade
storage_bucket text not null default 'run-screenshots'
storage_path text not null
step_name text null
page_url text null
caption text null
created_at timestamptz not null default now()
```

Indexes

```
screenshots_application_run_id_idx on screenshots(application_run_id)
screenshots_run_step_id_idx on screenshots(run_step_id)
screenshots_candidate_id_idx on screenshots(candidate_id)
screenshots_job_link_id_idx on screenshots(job_link_id)
screenshots_created_at_idx on screenshots(created_at)
```

RLS

- Admin/operator/viewer can view metadata.
 - Actual images accessed through signed URLs only.
 - Worker uploads using service role.
-

26. Table: automation_logs

Purpose

Stores technical logs from the worker.

Columns

```
id uuid primary key default gen_random_uuid()
application_run_id uuid null references application_runs(id) on delete cascade
run_step_id uuid null references run_steps(id) on delete set null
level text not null check level in ('debug', 'info', 'warn', 'error')
message text not null
context jsonb null
error_code text null
created_at timestamptz not null default now()
```

Indexes

```
automation_logs_application_run_id_idx on automation_logs(application_run_id)
automation_logs_run_step_id_idx on automation_logs(run_step_id)
automation_logs_level_idx on automation_logs(level)
automation_logs_created_at_idx on automation_logs(created_at)
```

RLS

- Admin/operator can view logs.
 - Viewer can view limited logs.
 - Sensitive values must be redacted before logging.
-

27. Table: system_settings

Purpose

Stores system-level configuration.

Columns

```
id uuid primary key default gen_random_uuid()
setting_key text not null unique
setting_value jsonb not null
description text null
is_sensitive boolean not null default false
updated_by uuid references users(id) null
created_at timestamptz not null default now()
updated_at timestamptz not null default now()
```

Required Settings

```
dry_run_required = true
human_approval_required = true
auto_submit_enabled = false
max_worker_concurrency = 1
otp_timeout_seconds = 120
default_run_mode = dry_run
```

RLS

- Admin can read/write settings.
- Operator can read non-sensitive settings.
- Viewer can read non-sensitive settings.
- Sensitive settings should not be exposed to frontend.

28. Table: audit_logs

Purpose

Stores important user actions for security and accountability.

Columns

```
id uuid primary key default gen_random_uuid()
actor_user_id uuid references users(id) null
action text not null
entity_type text not null
entity_id uuid null
candidate_id uuid null references candidates(id) on delete set null
application_run_id uuid null references application_runs(id) on delete set null
metadata jsonb null
ip_address text null
user_agent text null
created_at timestamptz not null default now()
```

Actions to Log

```
candidate_created
candidate_updated
resume_uploaded
resume_replaced
zoho_connected
job_link_added
dry_run_started
manual_review_approved
manual_review_edited
manual_review_marked_unsafe
final_submit_approved
application_submitted
settings_changed
```

Indexes

```
audit_logs_actor_user_id_idx on audit_logs(actor_user_id)
audit_logs_action_idx on audit_logs(action)
audit_logs_entity_type_idx on audit_logs(entity_type)
audit_logs_candidate_id_idx on audit_logs(candidate_id)
audit_logs_application_run_id_idx on audit_logs(application_run_id)
audit_logs_created_at_idx on audit_logs(created_at)
```

29. Storage Buckets

Use Supabase Storage.

All buckets must be private.

Required buckets:

```
candidate-resumes  
run-screenshots  
playwright-traces  
html-captures
```

30. Bucket: candidate-resumes

Purpose

Stores uploaded candidate resumes.

Path Structure

```
candidate-resumes/{candidate_id}/{resume_id}/{file_name}
```

Access

- Upload: admin/operator
- Read: admin/operator/viewer through signed URL
- Worker read: service role
- Public access: no

31. Bucket: run-screenshots

Purpose

Stores screenshots captured during automation.

Path Structure

```
run-screenshots/{application_run_id}/{step_order}-{step_name}.png
```

Access

- Upload: worker service role
 - Read: authenticated users through signed URL
 - Public access: no
-

32. Bucket: playwright-traces

Purpose

Stores Playwright traces for debugging.

Path Structure

```
playwright-traces/{application_run_id}/trace.zip
```

Access

- Upload: worker service role
 - Read: admin/operator through signed URL
 - Viewer: no by default
 - Public access: no
-

33. Bucket: html-captures

Purpose

Stores captured Workday HTML used for question extraction.

Path Structure

```
html-captures/{application_run_id}/{step_name}.html
```

Access

- Upload: worker service role
 - Read: admin/operator only
 - Viewer: no by default
 - Public access: no
-

34. Sensitive Fields That Need Encryption

The following fields must be encrypted before storing:

```
zoho_mailboxes.access_token_encrypted  
zoho_mailboxes.refresh_token_encrypted  
workday_accounts.password_encrypted
```

Optional encryption:

```
candidate_answers.answer_value for high-risk categories  
mapped_answers.final_answer for high-risk categories
```

Use application-layer encryption:

```
AES-256-GCM
```

Do not expose encryption key to frontend.

Environment variable:

```
ENCRYPTION_KEY
```

35. Row Level Security Rules

Enable RLS on all user-facing tables.

Tables requiring RLS:

```
users
candidates
candidate_resumes
candidate_answers
question_bank
zoho_mailboxes
workday_accounts
job_links
application_runs
run_steps
extracted_questions
mapped_answers
manual_review_items
otp_logs
screenshots
automation_logs
system_settings
audit_logs
```

36. RLS Summary

Admin

Can read and write most operational data.

Operator

Can create and manage candidates, resumes, job links, dry-runs, and manual review items.

Cannot change system-level security settings unless allowed.

Viewer

Can read non-sensitive operational data.

Cannot write.

Worker

Uses Supabase service role key.

Worker can:

- read queued jobs

- update run statuses
- write run steps
- write logs
- upload screenshots
- write extracted questions
- write mapped answers
- create manual review items

Worker must not run from frontend.

37. Worker Access Model

The Playwright worker should use:

```
SUPABASE_SERVICE_ROLE_KEY
```

The frontend should use:

```
EXPO_PUBLIC_SUPABASE_ANON_KEY
```

Important:

```
Never expose SUPABASE_SERVICE_ROLE_KEY to frontend.
```

38. API Endpoint List

Even though most frontend reads/writes can use Supabase directly, sensitive actions should go through backend/worker APIs.

Recommended endpoints:

38.1 Candidate APIs

```
POST /api/candidates
GET /api/candidates
GET /api/candidates/:id
```

```
PATCH /api/candidates/:id
POST /api/candidates/:id/archive
```

38.2 Resume APIs

```
POST /api/candidates/:id/resume/upload-url
POST /api/candidates/:id/resume/confirm-upload
POST /api/candidates/:id/resume/replace
GET /api/candidates/:id/resume/signed-url
```

38.3 Zoho APIs

```
GET /api/zoho/oauth/start
GET /api/zoho/oauth/callback
POST /api/zoho/test-otp-access
POST /api/zoho/disconnect
```

38.4 Job Link APIs

```
POST /api/job-links
POST /api/job-links/bulk
GET /api/job-links
GET /api/job-links/:id
PATCH /api/job-links/:id
POST /api/job-links/:id/skip
```

38.5 Run APIs

```
POST /api/job-links/:id/start-dry-run
GET /api/runs
GET /api/runs/:id
POST /api/runs/:id/retry
POST /api/runs/:id/stop
POST /api/runs/:id/continue
```

38.6 Manual Review APIs

```
GET /api/manual-review
GET /api/manual-review/:id
POST /api/manual-review/:id/approve
POST /api/manual-review/:id/edit
POST /api/manual-review/:id/mark-unsafe
POST /api/manual-review/:id/skip
POST /api/manual-review/:id/save-to-answer-bank
```

38.7 Submit APIs

```
POST /api/runs/:id/approve-final-submit
POST /api/runs/:id/submit
```

Important V1 rule:

```
/api/runs/:id/submit must only work after human approval.
```

38.8 File APIs

```
GET /api/screenshots/:id/signed-url
GET /api/traces/:runId/signed-url
GET /api/html-captures/:runId/signed-url
```

38.9 Settings APIs

```
GET /api/settings
PATCH /api/settings
GET /api/system/health
GET /api/worker/status
```

39. Webhooks and Event Triggers

V1 can start simple with worker polling.

Recommended V1:

```
Worker polls Supabase for queued job_links/application_runs.
```

Future option:

```
Database trigger or queue system
```

40. Worker Queue Logic

The worker should poll for queued runs:

```
Find application_runs where status = 'queued'  
Order by created_at asc  
Limit by WORKER_CONCURRENCY  
Lock or mark as starting  
Process run  
Write status updates
```

To avoid double-processing:

- update status from queued to starting atomically
 - use a database function or transaction
 - worker should only process runs it successfully claimed
-

41. Database Functions

Useful database functions:

```
claim_next_application_run()  
update_run_status()  
create_manual_review_item()
```

```
mark_review_item_resolved()  
calculate_run_readiness()
```

42. Function: claim_next_application_run

Purpose:

Worker safely claims one queued run.

Expected behavior:

```
Find oldest queued run  
Lock row  
Set status = starting  
Return run details
```

This prevents two workers from processing the same job.

43. Function: calculate_run_readiness

Purpose:

Calculate final dry-run readiness.

Possible output:

```
ready  
needs_review  
blocked  
failed
```

Rules:

```
If unresolved required manual review items exist → needs_review  
If failed critical step exists → blocked  
If all required answers filled and final page reached → ready  
Else → blocked
```

44. Event/Audit Triggers

Use triggers for:

```
updated_at refresh
audit important status changes
increment question_bank usage count
sync job_links.last_run_id
```

45. Status Flow

Job Link Status

```
queued
↓
running
↓
opened
↓
login_required
↓
otp_required
↓
logged_in
↓
resume_uploaded
↓
questionnaire_reached
↓
questions_extracted
↓
answers_mapped
↓
manual_review_required
↓
dry_run_complete
↓
approved_for_submit
↓
submitted
```

Failure states:

```
failed
duplicate
skipped
```

Application Run Status

```
queued
↓
starting
↓
opening_job_link
↓
detecting_page_state
↓
login_or_create_account
↓
waiting_for_otp
↓
otp_verified
↓
uploading_resume
↓
resume_uploaded
↓
reaching_questionnaire
↓
capturing_html
↓
extracting_questions
↓
mapping_answers
↓
filling_safe_answers
↓
checking_conditional_questions
↓
manual_review_required
↓
dry_run_complete
↓
human_approval_required
```

```
↓  
approved_for_submit  
↓  
submitting  
↓  
submitted
```

Failure states:

```
stopped  
failed
```

46. Risk Classification Rules

Every extracted question must be classified.

Risk Levels

```
low  
medium  
high  
unknown
```

Low Risk

Can be filled automatically if answer exists.

Examples:

```
name  
email  
phone  
address  
LinkedIn  
GitHub  
portfolio  
education  
employment history
```

Medium Risk

Can be filled only if answer exists and confidence is high.

Examples:

```
availability
notice period
preferred location
expected salary
relocation
```

High Risk

Must go to manual review.

Examples:

```
visa sponsorship
work authorization
disability
veteran status
gender
race
ethnicity
criminal history
background check consent
legal certification
```

Unknown

Always send to manual review.

47. Final Submit Safety Rule

In V1:

```
Auto-submit must be disabled by default.
```

The system may submit only when:

```
application_runs.status = approved_for_submit
approved_by is not null
approved_at is not null
readiness_score = ready
human_approval_required = true
```

Submit endpoint must reject request if these conditions are not met.

48. Data Retention

Recommended retention:

```
Automation logs: 90 days
Screenshots: 90 days or configurable
HTML captures: 30 days or configurable
Playwright traces: 30 days or configurable
Candidate data: until deleted or archived
Audit logs: 1 year minimum
```

Retention can be adjusted later based on company policy.

49. Security Checklist

The backend must ensure:

```
RLS enabled on all user-facing tables
Service role key only in worker
Anon key only in frontend
Private storage buckets
Signed URLs for private files
Encrypted Zoho tokens
Encrypted Workday passwords
No raw passwords in UI
No full OTP storage
No public resume access
No uncontrolled submit endpoint
Audit logs for important actions
Manual approval for high-risk answers
Dry-run required by default
```

50. Final Backend Recommendation

Build the backend around these core objects:

```
Candidate
Resume
Zoho Mailbox
Workday Account
Job Link
Application Run
Run Step
Extracted Question
Mapped Answer
Manual Review Item
Screenshot
Audit Log
```

The most important backend rule is:

```
The database must make unsafe automation impossible by default.
```

V1 should always follow this safety path:

```
Extract
↓
Map
↓
Fill safe answers
↓
Manual review if unsure
↓
Dry-run complete
↓
Human approval
↓
Controlled submit
```

Do not build backend shortcuts that allow final submission without review and approval.